LAB 4: WORKING WITH THREADS ON LINUX

I. Objectives

- Students understand the concept of thread, differences between thread and process, thread synchronization with mutex & condition variable and basic system calls when working with threads.

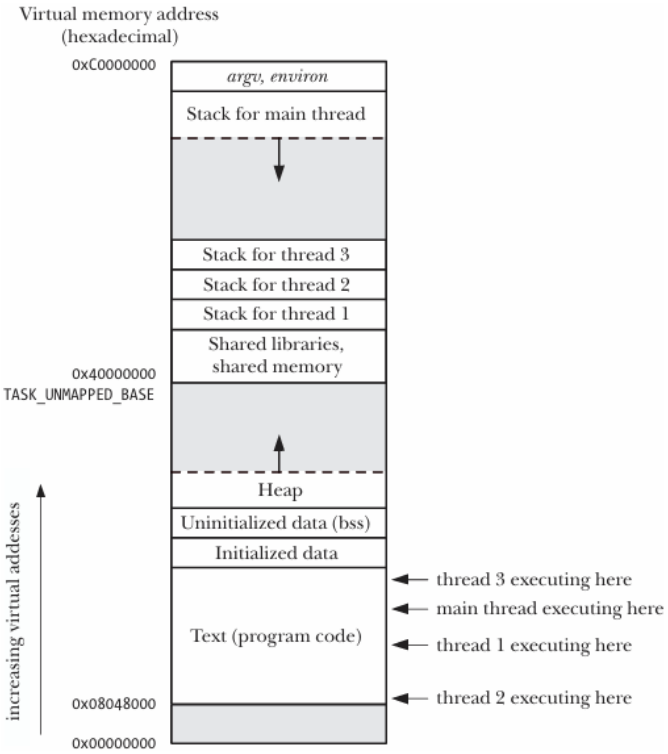
- Students can using system calls and library functions for requried tasks on threads handling.

II. Basic theory

1. Overview of thread

1.1 Introduction to thread

Like processes, threads are a mechanism that permits an application to perform multiple tasks concurrently. A single process can contain multiple threads. All of these threads are independently executing the same program, and they all share the same global memory, including the initialized data, uninitialized data, and heap segments.



The threads in a process can execute concurrently. On a multiprocessor system, multiple threads can execute parallel. If one thread is blocked on I/O, other threads are still eligible to execute.

In particular, the location of the per-thread stacks may be intermingled with shared libraries and shared memory regions, depending on the order in which threads are created, shared libraries loaded, and shared memory regions attached. Furthermore, the location of the per-thread stacks can vary depending on the Linux distribution.

Threads have some advantages over using multiple processes in applications that need to execute concurrently:

- Sharing information between threads is easy and fast. It is just a matter of copying data into shared (global or heap) variables.
- Thread creation is faster than process creation—typically, ten times faster or better. Thread creation is faster because many of the attributes that must be duplicated in a child created by fork() are instead shared between threads. In particular, copy-on-write duplication of pages of memory is not required, nor is duplication of page tables.

Besides global memory, threads also share a number of other attributes (i.e., these attributes are global to a process, rather than specific to a thread). These attributes include the following:

- process ID and parent process ID;
- process group ID and session ID;
- controlling terminal;
- process credentials (user and group IDs);
- open file descriptors;
- record locks created using fcntl();
- signal dispositions;
- file system–related information: umask, current working directory, and root directory;
- interval timers (setitimer()) and POSIX timers (timer_create());
- System V semaphore undo (semadj) values (Section 47.8);

- resource limits;
- CPU time consumed (as returned by times());
- resources consumed (as returned by getrusage()); and
- nice value (set by setpriority() and nice())

Among the attributes that are distinct for each thread are the following:

- thread ID;
- signal mask;
- thread-specific data;
- alternate signal stack (sigaltstack());
- the errno variable;
- floating-point environment (see fenv(3));
- realtime scheduling policy and priority;
- CPU affinity (Linux-specific,);
- capabilities (Linux-specific); and
- stack (local variables and function call linkage information)

1.2 Thread creation

When a program is started, the resulting process consists of a single thread, called the initial or main thread. In this section, we look at how to create additional threads.

The pthread_create() function creates a new thread.

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg);

Returns 0 on success, or a positive error number on error
```

Parameters:

- **thread**: A pointer to a pthread_t variable, where the new thread's ID is stored.
- **attr**: A pointer to a pthread_attr_t object that specifies the thread's attributes (can be NULL to use default attributes).
- **start**: A pointer to the function that the new thread will execute.
- **arg**: The argument passed to the start function.

Work:

- The new thread begins execution at the start() function, receiving the arg parameter.
- The thread that calls pthread_create() continues executing without waiting for the new thread to finish.
- The arg parameter is of type void *, allowing it to store a pointer to any data type (typically a pointer to a global or heap variable).
- If multiple arguments need to be passed, arg can be a pointer to a struct containing those arguments.

Considerations:

- Be cautious when using an **integer (int)** as the return value of the thread function, as it may conflict with PTHREAD_CANCELED (a special value returned when a thread is canceled).
- The **new thread's ID** is stored in **pthread_t**, but its initialization is not guaranteed before the new thread starts running.
- If the new thread needs to obtain its **own ID**, it must call **pthread_self()**.
- No guaranteed scheduling order exists between threads, so if execution order matters, synchronization mechanisms must be used.

1.3 Thread termination

A thread can terminate in one of the following ways:

- Returning from the start function: The thread's start function completes and returns a value.
- Calling pthread_exit(): The thread explicitly terminates using pthread_exit().
- Being canceled using pthread_cancel(): Another thread cancels it
- Process-wide termination:
    - Any thread calls exit(), terminating the entire process.
    - The main thread returns from main(), which terminates all threads immediately.

```
#include <pthread.h>

void pthread_exit(void *retval);
```

- .Terminates the calling thread.
- The retval parameter allows the thread to specify a return value.
- Another thread can retrieve this return value using pthread_join().

Considerations:

- Calling pthread_exit() is similar to returning from the thread's start function but can be done from any function within the thread's execution.
- The retval should not be stored on the thread's stack, as the stack's memory becomes invalid upon thread termination and might be reused by a new thread.
- If the main thread calls pthread_exit(), the process remains running as long as other threads are still executing.
- If the main thread calls exit() or returns from main(), all threads terminate immediately.

1.4 Thread ID

Each thread within a process is uniquely identified by a thread ID. This ID is returned to the caller of pthread_create(), and a thread can obtain its own ID using pthread_self().

```
#include <pthread.h>

pthread_t pthread_self(void);
```
Returns the **thread ID** of the calling thread

Uses of Thread IDs

- Thread management: Many Pthreads functions use thread IDs to identify the target thread, such as:
  - pthread_join()
  - pthread_detach()
  - pthread_cancel()
  - pthread_kill()
- Data structure tagging: Thread IDs can be used to mark dynamic data structures, helping to:
  - Track which thread created or owns a resource.
  - Identify which thread should later process that data.

Comparing Thread IDs

To check if two thread IDs are the same, use pthread_equal():

```
#include <pthread.h>
int pthread_equal(pthread_t t1, pthread_t t2);
```
Returns a **nonzero** value if t1 and t2 are the same, otherwise returns 0

Example: Checking if the calling thread matches a stored thread ID

```
if (pthread_equal(tid, pthread_self()))
   printf("tid matches self\n");
```

Use pthread_equal() instead of "==":

- pthread_t is an opaque data type and its representation can vary across systems.
- On Linux, pthread_t is defined as an unsigned long, but on other systems, it may be a pointer or structure.

- SUSv3 does not guarantee that pthread_t is a scalar type, so direct comparisons (==) may not work portably.

Thread IDs in Linux vs Other Systems:

- On Linux, thread IDs are unique across processes.
- However, on other systems, thread IDs may not be unique outside their own process.
- Thread IDs may be reused after:
  - A thread is joined using pthread_join().
  - A detached thread terminates.
- Linux provides a system call gettid() that returns a kernel-assigned thread ID, which is similar to a process ID.

1.5 Joining with a terminated thread

The pthread_join() function allows a thread to wait for another thread to terminate. If the specified thread has already finished, pthread_join() returns immediately.

```
#include <pthread.h>
int pthread_join(pthread_t thread, void **retval);
Returns 0 on success or a positive error number on failure.
```

Parameters:

- **thread**: The thread ID of the thread to join.
- **retval**: A pointer to store the return value of the terminated thread (as passed in pthread_exit() or the return value of the thread function). Can be NULL if not needed.

Considerations:

- If pthread_join() is called on a thread that has already been joined, unpredictable behavior can occur. The ID might be reused by another thread, leading to unintended joins.

- Joining is necessary for non-detached threads. If a thread is not joined, it becomes a zombie thread, wasting system resources. If too many zombie threads accumulate, no new threads can be created.

Comparison: pthread_join() vs waitpid()

pthread_join() for threads is similar to waitpid() for processes but with important differences:

- Threads are peers:
  - Any thread in a process can call pthread_join() to join any other thread in the process.
  - Example: If Thread A creates Thread B, which creates Thread C, then: Thread A can join with C, or Thread C can join with A.
  - This is different from processes, where only the parent process can wait() on its child.
- No **"join any thread"** option:
  - Unlike processes (where waitpid(-1, &status, options) can wait for any child process), pthread_join() only joins with a specific thread ID.
  - There is also no non-blocking join (unlike waitpid(WNOHANG), which checks without blocking). Workaround: Similar behavior can be achieved using condition variables.

1.6 Detaching a thread

By default, a thread is joinable, meaning that when it terminates, another thread can retrieve its return status using pthread_join(). However, in some cases, we don't need the thread's return status and simply want the system to automatically clean up the thread when it terminates. In such cases, we can mark the thread as detached using pthread_detach().

```
#include <pthread.h>
int pthread_detach(pthread_t thread);
Returns 0 on success or a positive error number on failure
```

Parameters:

- **thread**: The thread ID of the thread to join.

Considerations:

- Once a thread is detached, it cannot be joined using pthread_join(), and its return status cannot be retrieved.
- A detached thread cannot be made joinable again.
- Detaching a thread does not prevent it from being terminated if:
  - Another thread calls exit(), or
  - The main thread returns from main().
- In these cases, all threads in the process (including detached threads) terminate immediately.
- pthread_detach() only controls what happens after a thread terminates, but it does not affect how or when the thread terminates.

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *threadFunc(void *arg) {
   printf("Hello from thread!\n");
   return NULL;
}

int main() {
   pthread_t thread;

   // Create a thread
   pthread_create(&thread, NULL, threadFunc, NULL);

   printf("Hello from main!\n");
```

```
    // Wait for the thread to finish
    pthread_join(thread, NULL);

    return 0;
}
```

Note: GCC does not automatically link the pthread library so you need to explicitly link it (using -pthread flag) when compile your program.

gcc -pthread example.c -o example

1.7 Thread attributes

When creating a thread using pthread_create(), the attr argument (of type pthread_attr_t) can be used to customize the thread's attributes.

Although we won't go into full details, these attributes include:

- **Stack location and size**: Specifies where and how large the thread's stack should be.
- Scheduling policy and priority: Similar to process scheduling policies.
- Joinable or detached status: Determines whether the thread will need to be joined later (joinable) or if it will clean up itself automatically (detached).

Example: Creating a Detached Thread

Instead of detaching a thread after creation using pthread_detach(), a thread can be created already detached by setting the appropriate attribute in pthread_attr_t.

Process:

- Initialize a pthread_attr_t structure with default values.
- Modify the attribute to specify that the thread should be detached.
- Use pthread_create() while passing the modified attributes.
- Once the thread is created, destroy the pthread_attr_t object since it is no longer needed.

This method allows better control over thread behavior at creation time rather than modifying attributes later.

2. Thread synchronization with mutex

2.1 Introduction to mutex

To avoid the problems that can occur when threads try to update a shared variable, we must use a mutex (short for mutual exclusion) to ensure that only one thread at a time can access the variable. More generally, mutexes can be used to ensure atomic access to any shared resource, but protecting shared variables is the most common use.

A mutex has two states: **locked** and **unlocked**. At any moment, at most one thread may hold the lock on a mutex. Attempting to lock a mutex that is already locked either blocks or fails with an error, depending on the method used to place the lock. When a thread locks a mutex, it becomes the owner of that mutex. Only the mutex owner can unlock the mutex.

In general, we employ a different mutex for each shared resource (which may consist of multiple related variables), and each thread employs the following protocol for accessing a resource:

- lock the mutex for the shared resource;
- access the shared resource; and
- unlock the mutex.

If multiple threads try to execute this block of code (a critical section), the fact that only one thread can hold the mutex (the others remain blocked) means that only one thread at a time can enter the block, as illustrated in below figure:

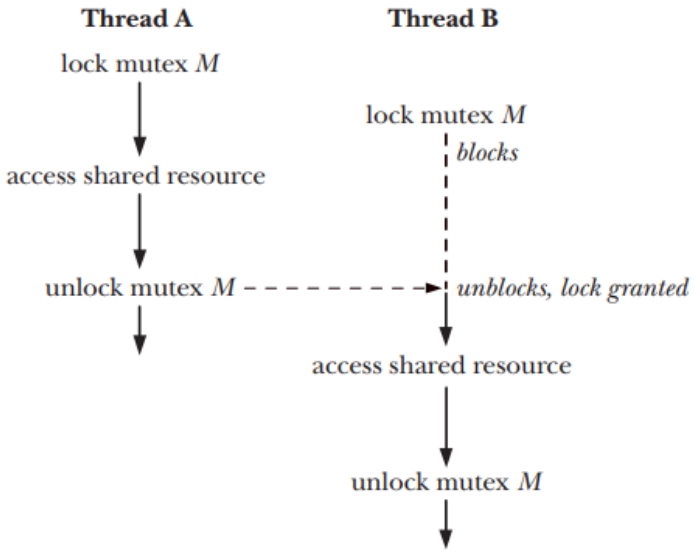


Figure 30-2: Using a mutex to protect a critical section

2.2 Allocated mutexes

A mutex is a variable of the type *pthread_mutex_t*. Before it can be used, a mutex must always be initialized. A mutex can either be allocated as a static variable or be created dynamically.

Statically Allocated Mutexes

A statically allocated mutex is declared as a global or static variable and is initialized at compile time. It is initialized using PTHREAD_MUTEX_INITIALIZER at compile time and available for the lifetime of the program.

Example:

```
#include <pthread.h> #include <stdio.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Statically allocated
```

Dynamically Allocated Mutex

A dynamically allocated mutex is created at runtime using *malloc()* and explicitly initialized using *pthread_mutex_init()* (The static initializer value PTHREAD_MUTEX_INITIALIZER can be used only for initializing a statically allocated mutex with default attributes).

```
#include <pthread.h>
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
Returns 0 on success, or a positive error number on error
```

The mutex argument identifies the mutex to be initialized. The *attr* argument is a pointer to a *pthread_mutexattr_t* object that has previously been initialized to define the attributes for the mutex. If *attr* is specified as NULL, then the mutex is assigned various default attributes.

2.3 Locking and unlocking a mutex

After initialization, a mutex is unlocked. To lock and unlock a mutex, we use the *pthread_mutex_lock()* and *pthread_mutex_unlock()* functions.

```
#include <pthread.h>
int pthread_mutex_lock (pthread_mutex_t *mutex);
int pthread_mutex_unlock (pthread_mutex_t *mutex);
Both return 0 on success, or a positive error number on error
```

To lock a mutex, we specify the mutex in a call to *pthread_mutex_lock()*. If the mutex is currently unlocked, this call locks the mutex and returns immediately. If the mutex is currently locked by another thread, then *pthread_mutex_lock()* blocks until the mutex is unlocked, at which point it locks the mutex and returns.

If the calling thread itself has already locked the mutex given to *pthread_mutex_lock(), then*, for the default type of mutex, one of two implementation defined possibilities may result: the thread deadlocks, blocked trying to lock a mutex that it already owns, or the call fails, returning the error EDEADLK. On Linux, the thread deadlocks by default.

The *pthread_mutex_unlock()* function unlocks a mutex previously locked by the calling thread. It is an error to unlock a mutex that is not currently locked, or to unlock a mutex that is locked by another thread. (When a thread locks a mutex, it becomes the owner of that mutex. Only the mutex owner can unlock the mutex.) If more than one other thread is waiting to acquire the mutex unlocked by a call to *pthread_mutex_unlock()*, it is indeterminate which thread will succeed in acquiring it.

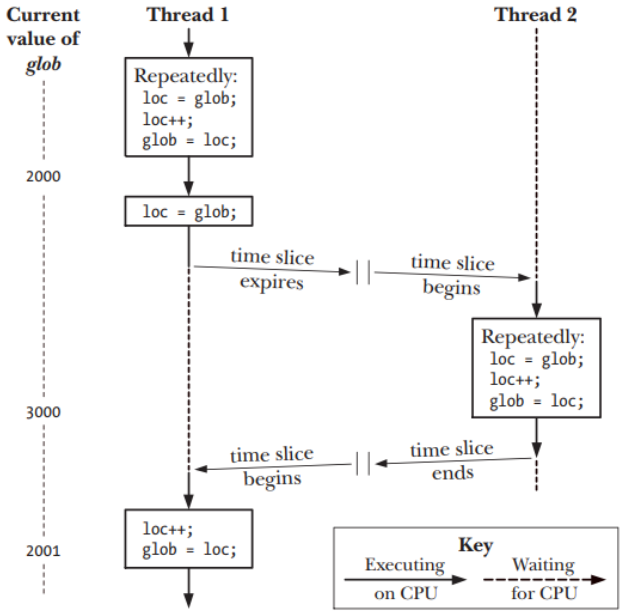


Figure 30-1: Two threads incrementing a global variable without synchronization

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

static int glob = 0;
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static void *           /* Loop 'arg' times incrementing 'glob' */
threadFunc(void *arg) {
int loops = *((int *) arg);
int loc, j, s;
for (j = 0; j < loops; j++) {
s = pthread_mutex_lock(&mtx);
if (s != 0)
errExitEN(s, "pthread_mutex_lock");
loc = glob;
loc++;
glob = loc;
```

```
s = pthread_mutex_unlock(&mtx);
 if (s != 0)
errExitEN(s, "pthread_mutex_unlock");
}
return NULL; }
int main(int argc, char *argv[]) {
pthread_t t1, t2;
int loops, s;
loops = 10000000;
s = pthread_create(&t1, NULL, threadFunc, &loops);
if (s != 0)
errExitEN(s, "pthread_create");
s = pthread_create(&t2, NULL, threadFunc, &loops);
if (s != 0)

errExitEN(s, "pthread_create");
s = pthread_join(t1, NULL);
if (s != 0)
errExitEN(s, "pthread_join");
s = pthread_join(t2, NULL);
if (s != 0)
errExitEN(s, "pthread_join");
printf("glob = %d\n", glob);
 exit(EXIT_SUCCESS);
}
```

2.4 Destroying mutexes

When an automatically or dynamically allocated mutex is no longer required, it should be destroyed using *pthread_mutex_destroy()*. (It is not necessary to call *pthread_mutex_destroy()* on a mutex that was statically initialized using PTHREAD_MUTEX_INITIALIZER.)

```
#include <pthread.h>
int pthread_mutex_destroy (pthread_mutex_t *mutex);
Returns 0 on success, or a positive error number on error
```

It is safe to destroy a mutex only when it is unlocked, and no thread will subsequently try to lock it. If the mutex resides in a region of dynamically allocated memory, then it should be destroyed before freeing that memory region. An automatically allocated mutex should be destroyed before its host function returns.

A mutex that has been destroyed with *pthread_mutex_destroy()* can subsequently be reinitialized by *pthread_mutex_init().*

3. Condition variables

A mutex prevents multiple threads from accessing a shared variable at the same time. A condition variable allows one thread to inform other threads about changes in the state of a shared variable (or other shared resource) and allows the other threads to wait (block) for such notification.

A condition variable is always used in conjunction with a mutex. The mutex provides mutual exclusion for accessing the shared variable, while the condition variable is used to signal changes in the variable's state.

3.1 Allocated Condition Variables

A condition variable has the type pthread_cond_t. A condition variable must be initialized before use.

Statically allocated condition variables

A statically allocated condition variable is declared as a global or static variable and initialized at compile time. A static condition variable can be initialized using the PTHREAD_COND_INITIALIZER macro:

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

This ensures that the condition variable is ready to use without requiring explicit initialization.

Dynamically allocated condition variables

A dynamically allocated condition variable is created at runtime using *malloc()* and explicitly initialized with *pthread_cond_init().* Using

pthread_cond_init() we can initialize automatically and dynamically allocated condition variables, and to initialize a statically allocated condition variable with attributes other than the defaults.

```
#include <pthread.h>
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
Returns 0 on success, or a positive error number on error
```

The *cond* argument identifies the condition variable to be initialized. As with mutexes, we can specify an *attr* argument that has been previously initialized to determine attributes for the condition variable. Various Pthreads functions can be used to initialize the attributes in the *pthread_condattr_t* object pointed to by *attr*. If *attr* is NULL, a default set of attributes is assigned to the condition variable.

3.2 Signaling and Waiting on Condition Variables

The principal condition variable operations are **signal** and **wait**. The signal operation is a notification to one or more waiting threads that a shared variable's state has changed. The wait operation is the means of blocking until such a notification is received.

Signaling a Condition Variable

A thread signals a condition variable when it changes the state of a shared resource and wants to notify waiting threads.

```
int pthread_cond_signal (pthread_cond_t *cond);
int pthread_cond_broadcast (pthread_cond_t *cond);
```

Parameter:

- cond: The condition variable to wait on.

Return value: return 0 on success, or a positive error number on error

The difference between *pthread_cond_signal()* and *pthread_cond_broadcast()* lies in what happens if multiple threads are blocked (wait) in *pthread_cond_wait()*. With *pthread_cond_signal()*, we are simply guaranteed that at least one of the blocked threads is woken up; with *pthread_cond_broadcast()*, all blocked threads are woken up.

Waiting on a condition variable

A thread waits on a condition variable when it needs to be notified before proceeding. While waiting:

- The mutex is automatically unlocked so other threads can access and modify the shared resource.
- The thread is put into a blocked state (sleeps) until another thread signals it.
- Once signaled, it relocks the mutex and continuing execution, thread then immediately accesses the shared variable.

The *pthread_cond_wait()* function automatically performs the mutex unlocking and locking.

```
#include <pthread.h>
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

Parameter:

- cond: The condition variable to wait on.
- mutex: The mutex protecting the shared resource.

Return value: return 0 on success, or a positive error number on error

Example:

```
#include<stdio.h>
#include<pthread.h>
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int ready = 0; // Shared condition variable

void *prodfun(void *arg)
{
pthread_mutex_lock(&lock);
ready = 1;
printf("Producer: Ready, signal consumer\n");
pthread_mutex_unlock(&lock);
pthread_cond_signal(&cond);
}

void *consfun(void *arg){
pthread_mutex_lock(&lock);
while (!ready){
printf("Consumer: Waiting...\n"); pthread_cond_wait(&cond, &lock);
```

```
}
printf("Consumer: Data received.\n");
pthread_mutex_unlock(&lock);
}


void main()
{pthread_t prod, cons;
pthread_create(&prod, NULL, prodfun, NULL);
pthread_create(&cons, NULL, consfun, NULL);
pthread_join(prod, NULL);
pthread_join(cons, NULL);
pthread_mutex_destroy(&lock);
pthread_cond_destroy(&cond);
}
```

3.3 Destroying condition variable

When an automatically or dynamically allocated condition variable is no longer required, then it should be destroyed using *pthread_cond_destroy()*. It is not necessary to call *pthread_cond_destroy()* on a condition variable that was statically initialized using PTHREAD_COND_INITIALIZER.

```
#include <pthread.h>
int pthread_cond_destroy(pthread_cond_t *cond);
Returns 0 on success, or a positive error number on error
```

It is safe to destroy a condition variable only when no threads are waiting on it. If the condition variable resides in a region of dynamically allocated memory, then it should be destroyed before freeing that memory region. An automatically allocated condition variable should be destroyed before its host function returns. A condition variable that has been destroyed with *pthread_cond_destroy()* can subsequently be reinitialized by *pthread_cond_init().*